Diseño e Implementación de Contadores

Herrera Mauro Leandro

RESUMEN

El presente informe explica cómo se diseñaron, sintetizaron y simularon circuitos contadores sincrónicos usando diferentes métodos. Primero, se desarrolla un contador binario de tres bits con reset sincrónico usando el método de máquinas de estado finito basado en estados y transiciones. Después, se implementa un contador binario de cuatro bits con entradas de reset, carga y habilitación usando la técnica de datapath, con un sumador de 4 bits que tiene propagación de acarreo. En ambos diseños se muestran los circuitos sintetizados y se estudian sus diagramas de tiempo. Por último, se describe en VHDL un contador de segundos de un dígito decimal para un display de siete segmentos que funciona con un oscilador de 12 MHz. También se incluye su banco de pruebas para validar el hardware antes de implementarlo en FPGA.

1 INTRODUCCIÓN

Un contador es un circuito secuencial formado por varios flip-flops conectados entre sí. Sirve para contar al pasar por una serie de estados ya definidos. En los sistemas digitales, funcionan como registros que siguen una secuencia numérica específica, llamada módulo, cuando reciben pulsos de entrada. Según si los flip-flops reciben el mismo pulso de reloj o no, estos dispositivos pueden ser síncronos o asíncronos. Por otro lado, el diseño usando la técnica de datapath, o ruta de datos, consiste en dividir la microarquitectura de un sistema en dos partes que trabajan juntas: la lógica de control y la ruta de datos. El datapath reúne todo el hardware encargado de operar y guardar información, como memorias, registros, multiplexores y sumadores. La ruta de datos cambia los valores según lo que pide el sistema. La lógica de control

es un circuito secuencial aparte que le da las órdenes necesarias para coordinar cada paso del proceso.

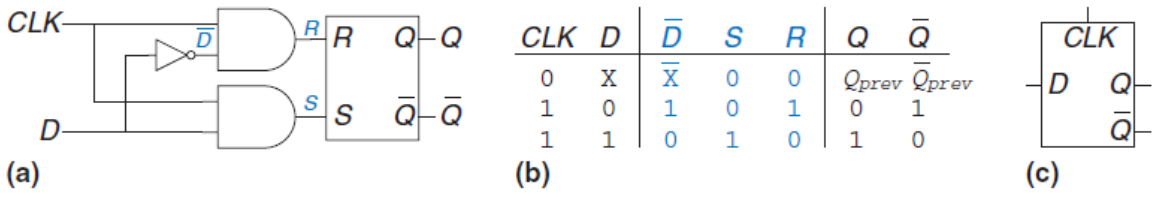
2 MATERIALES Y METODOS

Los conceptos desarrollados a continuación son una síntesis de la investigación bibliográfica realizada.

2.1 CERROJO D

El cerrojo D (ver figura 1) resuelve el problema del cerrojo RS, que tenía una indeterminación cuando ambas entradas eran HIGH. Este circuito consta de dos entradas: La entrada de datos D que controla el siguiente estado en el debería estar y la entrada de reloj, denotado con CLK (del inglés, *clock*), el cual controla cuándo el estado debería cambiar.

Por conveniencia, se consideran los nodos internos $\bar{D}$, S y R. Si el reloj se encuentra en bajo, CLK=0, S y R son FALSE sin importar el valor de D. Si CLK=1, una compuerta NAND producirá una salida verdadera y la otra falsa, dependiendo del valor de D. Dados S y R, quedan determinados Q y $\bar{Q}$ de acuerdo al funcionamiento del circuito RS, sin la indeterminación, pues siempre se cumple que Q es el complemento de $\bar{Q}$. En resumen, en el circuito cerrojo D, Q cambiará al mismo estado que D si y sólo si el reloj CLK=1, en caso contrario mantendrá su estado previo sin importar el valor de D. (Harris and Harris, (2019), pp. 111)



CLK	D	$\bar{D}$	S	R	Q	$\bar{Q}$
0	X	$\bar{X}$	0	0	Q_{prev}	$\bar{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Figura 1 *Cerrojo D: (a) Esquemático (b) Tabla de verdad (c) Símbolo (Harris and Harris, (2019), pp. 111)*

2.2 FLIP-FLOP D MAESTRO-ESCLAVO

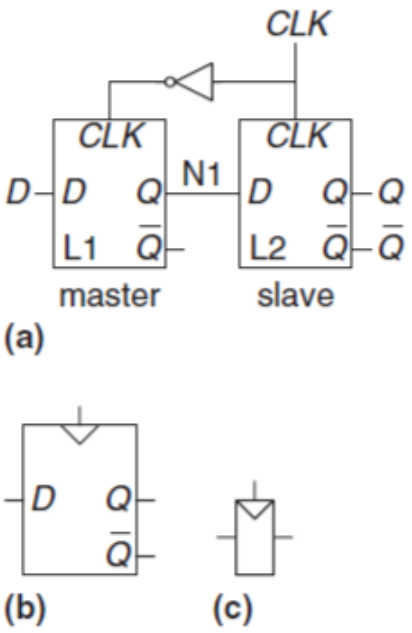


Figura 2 Flip-Flop D: (a) Esquemático (b) Símbolo (c) Símbolo reducido
(Harris y Harris, (2019), pp. 112)

El flip-flop D puede ser armado a partir de dos cerrojos D en oposición controlados por relojes complementarios, como se muestra en la Figura 2. El primer cerrojo L1 se llama líder y el segundo L2 se llama seguidor, porque hace lo que haga L1.

Se ha llamado N1 al nodo entre ambos cerrojos. Un símbolo para el flip-flop D se muestra en la Figura 2(b). Cuando la salida $\bar{Q}$ no es requerida, se suele usar el símbolo reducido, ver Figura 2 (c).

En términos simples, un flip-flop D copia el estado de D en Q durante el flanco ascendente del reloj y recuerda ese estado para cualquier otro momento. El flanco ascendente del reloj suele llamarse *flanco de reloj*, por abreviación. La entrada D especifica qué nuevo valor tendrá Q y el flanco del reloj indicará cuándo deberá actualizarse. (Harris y Harris, (2019), pp. 112)

2.3 REGISTRO

Un registro de N bits es un banco de N flip-flops que comparten entrada de reloj así las actualizaciones se realizan al mismo tiempo. Son el bloque principal de la mayoría de los circuitos secuenciales.

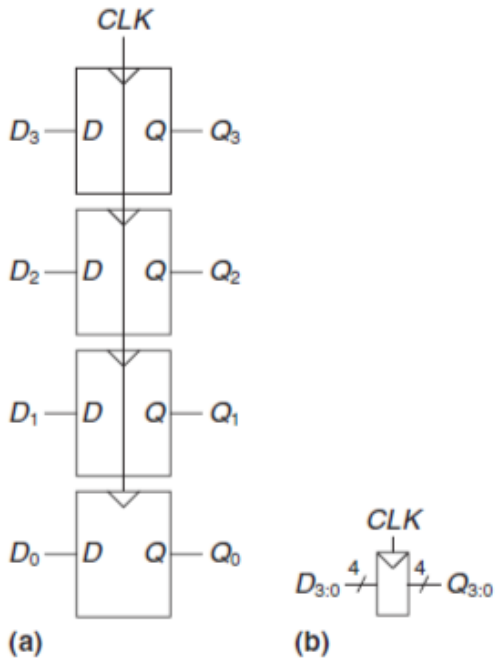


Figura 3 *Registro de 4 bits: (a)Esquema (b) Símbolo (Harris y Harris, (2019), pp. 112)*

2.4 CIRCUITO SECUENCIAL

Un circuito secuencial es un tipo de circuito digital cuya salida no solo depende de las entradas actuales, sino también del estado anterior del sistema. Es decir, tienen realimentación y las salidas no están necesariamente definidas por una tabla de verdad. Esto lo distingue de los circuitos combinacionales, donde las salidas dependen únicamente de las entradas en un momento dado y quedan definidas por una tabla de verdad.

Los circuitos secuenciales tienen memoria interna, lo que les permite recordar estados previos y, en función de eso, determinar la salida. (Harris y Harris, (2019), pp. 117)

2.5 CIRCUITO SECUENCIAL ASINCRÓNICO

Un circuito secuencial asincrónico es un tipo de circuito secuencial en el que los cambios de estado ocurren de manera inmediata cuando las entradas cambian, sin estar sincronizados por un reloj o una señal de sincronización global. (Harris y Harris, (2019), pp. 120-121)

2.6 CIRCUITO SECUENCIAL SINCRÓNICO

Un circuito secuencial sincrónico es un tipo de circuito secuencial en el que los cambios de estado y las salidas ocurren en sincronización con una señal de reloj. A diferencia de los circuitos asincrónicos, donde los cambios de estado dependen directamente de las entradas, los circuitos sincrónicos dependen de una señal de reloj que regula cuándo los cambios pueden ocurrir, lo que proporciona una operación más estable y predecible. (Harris y Harris, (2019), pp. 119)

2.7 MÁQUINAS DE ESTADO FINITO

Concepto

Una máquina de estado finito (FSM, por sus siglas en inglés: *Finite State Machine*) es un modelo matemático utilizado para describir el comportamiento de un sistema que puede estar en uno de un conjunto finito de estados, y que cambia de un estado a otro en respuesta a ciertas entradas. Son un subconjunto de los circuitos sincrónicos.

Estructura general y tipos

Su nombre es debido a que un circuito con k registros puede estar en un número finito (2^k) de estados únicos. Una FSM tiene M entradas, N salidas y k bits de estados. Recibe la señal de un reloj y, opcionalmente, la señal de un RESET.

Existen dos tipos de máquinas de estado finito (ver Figura 4), caracterizadas por su funcionamiento:

- **Máquina de Moore:** En este tipo de FSM, las salidas dependen únicamente del estado actual. Es decir, la salida es constante mientras el sistema se encuentra en un estado particular.
- **Máquina de Mealy:** En una FSM de Mealy, las salidas dependen tanto del estado actual como de las entradas. Esto permite que las salidas cambien de manera más rápida en respuesta a las entradas.

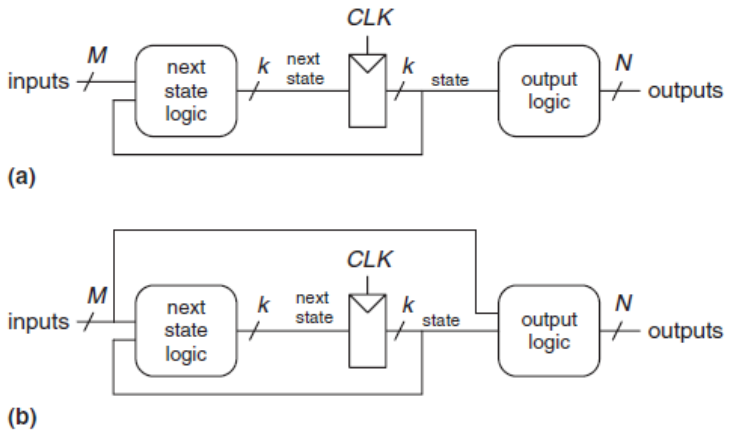


Figura 4 *Maquina de estado finito: (a) Máquina de Moore (b) Máquina de Mealy (Harris y Harris, (2019), pp. 121*

Formas de representación

Para facilitar su diseño, análisis y comprensión, las máquinas de estado finito se suelen representar mediante dos herramientas principales:

- **Diagrama de estados (o diagrama de transición de estados):** Es una representación gráfica intuitiva. Consiste en nodos (círculos) que representan cada uno de los estados posibles del sistema, conectados por flechas dirigidas que indican las transiciones entre

ellos. Junto a las flechas se anotan las condiciones lógicas de entrada que desencadenan el cambio. En las máquinas de Moore, la salida se anota dentro del nodo del estado, mientras que en las de Mealy se anota sobre la flecha de transición.

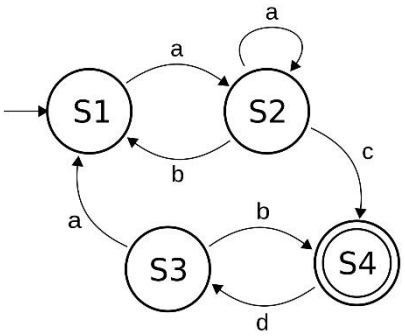


Figura 5. *Representación gráfica de una máquina de estado mediante un diagrama de transición de estados.*

Tabla de transición de estados: Es una representación en forma de matriz o tabla (similar a una tabla de verdad) que lista todas las combinaciones posibles del estado actual y las entradas del sistema. Para cada combinación, indica cuál será el estado siguiente (Next State) y cuáles serán los valores de las señales de salida.

Estados	Transición			Salida		
	Entrada			Entrada		
	a	b	c	a	b	c
s_0	s_0	s_3	s_2	0	1	1
s_1	s_1	s_1	s_3	0	0	1
s_2	s_1	s_1	s_3	1	1	0
s_3	s_2	s_3	s_0	1	0	1

Tabla 1. Tabla de transición de estados detallando el estado siguiente y las salidas en función del estado actual y las entradas.

Asignación de estados (Codificación)

Para implementar una FSM en hardware físico mediante flip-flops, cada estado simbólico (por ejemplo, S_0, S_1, S_2) debe traducirse a un código binario. Este proceso se llama asignación de estados, y los dos métodos más comunes son:

- **Codificación binaria (Binary Encoding):** Utiliza el número mínimo de bits matemáticamente necesarios para representar todos los estados (por ejemplo, para 3 estados se usan 2 bits: 00, 01, 10). Su ventaja es que minimiza la cantidad de flip-flops que el circuito requiere.
- **Codificación One-Hot:** Utiliza un flip-flop (un bit) individual para cada estado. En cualquier momento dado, solo un bit está activo (en ALTO o "hot") y el resto en BAJO. Por ejemplo, para tres estados la codificación sería 001, 010, 100. Aunque requiere más registros (mayor uso de hardware), simplifica considerablemente la lógica combinacional para el próximo estado, lo que a menudo permite que el circuito sea más rápido.

Análisis y síntesis

Son los dos enfoques metodológicos opuestos que se utilizan al trabajar con circuitos secuenciales:

- **Análisis:** Es el proceso de ingeniería inversa. Consiste en tomar un diagrama de circuito secuencial ya diseñado (que incluye registros y compuertas lógicas) y determinar su comportamiento. A partir del circuito, se extraen las ecuaciones booleanas, se reconstruye la tabla de transición y finalmente se dibuja el diagrama de estados para entender qué hace el sistema.

- **Síntesis (o diseño):** Es el proceso de crear un circuito secuencial desde cero a partir de la especificación de un problema. Los pasos estructurados de la síntesis son:
    1. Interpretar el problema y dibujar el diagrama de estados.
    2. Volcar la información en una tabla de transición de estados.
    3. Elegir y aplicar una asignación de estados (codificación binaria o one-hot).
    4. Determinar la tabla de verdad para la lógica del estado siguiente y de la salida.
    5. Simplificar las ecuaciones lógicas (utilizando métodos algebraicos o mapas de Karnaugh).
    6. Implementar (dibujar el esquemático de) el circuito combinando los registros y las compuertas lógicas calculadas

2.8 Diseño de Circuitos Secuenciales mediante la técnica de Datapath (Ruta de Datos)

El diseño lógico de un sistema digital complejo se divide en dos partes fundamentales: el diseño de los circuitos que procesan los datos (la **ruta de datos** o *datapath*) y el diseño de los circuitos que determinan el orden de las operaciones (la **lógica de control**).

La técnica de diseño por *datapath* es un enfoque estructurado y modular para implementar algoritmos en hardware. Esta técnica divide la microarquitectura de un sistema en los dos subsistemas mencionados:

1. La Ruta de Datos (*Datapath*): Es el subsistema encargado de almacenar, manipular y procesar los elementos discretos de información. Consiste en una interconexión de bloques

funcionales (o módulos) que operan sobre los datos (comúnmente organizados en "buses" o palabras de varios bits). Los componentes típicos del datapath incluyen:

- **Registros y Memorias:** Elementos secuenciales sincronizados por un reloj que almacenan el estado del sistema o datos temporales.
- **Unidades Aritméticas (ALU, sumadores, restadores):** Circuitos combinacionales que realizan operaciones matemáticas y lógicas sobre los datos.
- **Multiplexores:** Actúan como "interruptores" de ruta, permitiendo seleccionar qué datos de entrada fluirán hacia los registros o unidades de cálculo.

2. La Lógica de Control (Controlador): Es un circuito secuencial independiente (frecuentemente diseñado como una Máquina de Estado Finito) que supervisa y coordina las operaciones dentro de la ruta de datos. El controlador recibe información sobre el estado actual (por ejemplo, entradas externas o señales condicionales del datapath) y genera una secuencia temporal de señales de mando (comandos de habilitación, *reset*, o selección de multiplexores) que le indican a la ruta de datos exactamente qué tarea procesar en cada ciclo de reloj.

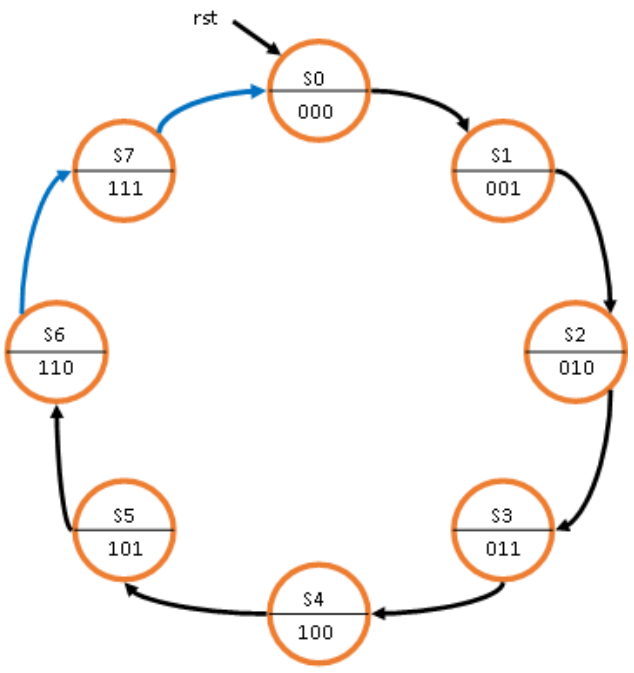
Ventajas del método: Al separar el flujo de la información (datapath) de la toma de decisiones (controlador), se simplifica radicalmente el diseño y manejo de la complejidad en los sistemas digitales. En lugar de tratar de especificar un sistema masivo con una tabla de estados infinita, el diseñador puede ensamblar de forma jerárquica y modular componentes estándar (como sumadores y registros) conectados por rutas lógicas claras.

3 RESULTADOS

Paso 1: Definir las entradas, salidas y estados

- **Estados:** Como es un contador binario de 3 bits, contará del 0 al 7. Necesitamos 8 estados (los llamaremos del S_0 al S_7) que codificaremos en binario desde 000 hasta 111.
- **Entradas:** Tendrá una única entrada de control llamada Reset (además del reloj CLK que sincroniza todo).
- **Salidas:** Las salidas serán los mismos bits de estado (Q_2, Q_1 y Q_0). Esto corresponde al modelo de una máquina de Moore, donde las salidas dependen únicamente del estado actual.

Paso 2: El Diagrama de Transición de Estados Condición de Conteo (Reset = 0): El contador avanza al siguiente número. Las flechas irán de 000 → 001 → 010 … → 111 y volverá a empezar en 000.**Condición de Reinicio (Reset = 1):** No importa en qué estado se encuentre el



contador, al llegar el flanco del reloj, el sistema debe saltar directamente al estado 000. Por lo tanto, habrá flechas desde todos los estados apuntando al 000 con la condición Reset=1.

Figura 6. Diagrama de transición de estados para el contador binario de 3 bits con reset sincrónico.

Paso 3: Tabla de Transición de Estados Con el comportamiento definido, volcamos todo a una tabla. Para simplificar el diseño en hardware, asumiremos el uso de Flip-Flops tipo D, donde el "Estado Siguiente" es exactamente igual a la entrada de datos (D_2, D_1 y D_0).

La estructura de la tabla se vería así:

Entrada (Reset)	**Estado Actual (Q_2, Q_1 y Q_0)**	**Estado Siguiente / Entradas (D_2, D_1 y D_0)**
1	X X X (No importa)	0 0 0
0	0 0 0	0 0 1
0	0 0 1	0 1 0
0	0 1 0	0 1 1
0	0 1 1	1 0 0
0	1 0 0	1 0 1
0	1 0 1	1 1 0
0	1 1 0	1 1 1
0	1 1 1	0 0 0

Tabla 2 - Tabla de transición de estados

Paso 4: Mapas de Karnaugh y ecuaciones lógicas Para implementar la lógica combinacional del próximo estado, se utiliza el método del mapa de Karnaugh, el cual ofrece un

procedimiento sencillo y directo para minimizar funciones booleanas. En este diseño, tenemos 4 variables de entrada para nuestra lógica combinacional: la señal Reset (R) y las tres salidas actuales de los registros (Q_2, Q_1 y Q_0).

Analizando la Tabla 2 se construyen los siguientes mapas de Karnaugh para D_2, D_1 y D_0.

D_2 R Q_2 / Q_1 Q_0	0 0	0 1	1 1	1 0
0 0	0	1	0	0
0 1	0	1	0	0
1 1	1	0	0	0
1 0	1	1	0	0

Tabla 3.1 - Mapa de Karnaugh para la entrada de datos **D_2**

Minimizando el mapa de Karnaugh mediante sumas de productos lógicos (min-términos), nos queda la siguiente expresión: $D_2 = \overline{RQ_1}Q_2 + Q_1Q_0\overline{RQ_2} + \overline{Q_0R}Q_2$

D_1 R Q_2 / Q_1 Q_0	0 0	0 1	1 1	1 0
0 0	0	0	0	0
0 1	1	1	0	0
1 1	0	0	0	0
1 0	1	1	0	0

Tabla 3.2 - Mapa de Karnaugh para la entrada de datos **D_1**

Minimizando el mapa de Karnaugh mediante sumas de productos lógicos (min-términos), nos queda la siguiente expresión: $D_1 = \overline{RQ_1}Q_0 + \overline{RQ_0}Q_1$

D_0 / Q_1 Q_0 \ R Q_2	0 0	0 1	1 1	1 0
0 0	1	1	0	0
0 1	0	0	0	0
1 1	0	0	0	0
1 0	1	1	0	0

Tabla 3.3 - Mapa de Karnaugh para la entrada de datos $\mathbf{D_0}$

Minimizando el mapa de Karnaugh mediante sumas de productos lógicos (min-términos), nos queda la siguiente expresión: $D_0 = \overline{RQ_0}$

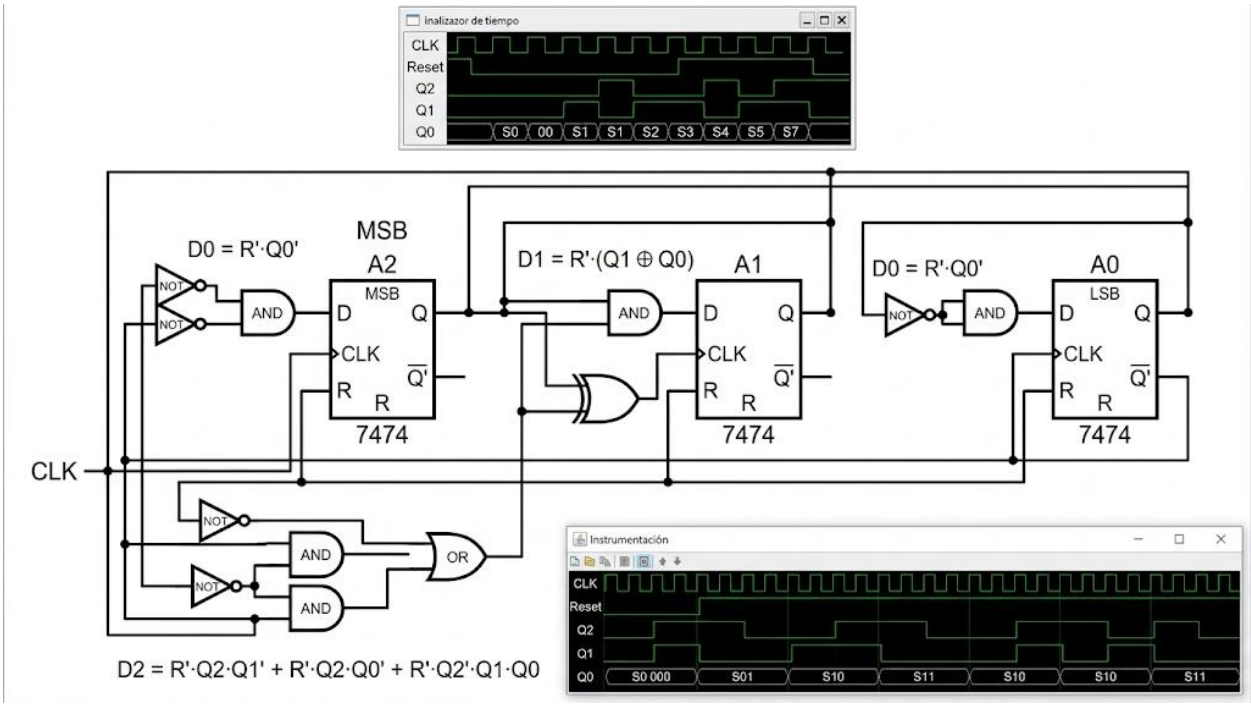


Figura 7. Circuito esquemático del contador binario de 3 bits con reset sincrónico y su correspondiente diagrama de tiempos (simulación de comportamiento).

4 Contador binario de 4 bits (Técnica Datapath)

Para implementar el contador binario de 4 bits con funciones de reset, carga (load) y habilitación de conteo (enable), se optó por la técnica de diseño *Datapath*. Esta técnica permitió dividir la arquitectura del circuito, evitando el uso de una máquina de estado finito (FSM) de 16 estados, lo cual resultaría impráctico.

El diseño se estructuró interconectando tres componentes fundamentales de hardware estándar:

1. **Registro de 4 bits (`registro4b`):** Actúa como la memoria del sistema, almacenando el estado actual del contador. Es estrictamente sincrónico, actualizando su salida únicamente en los flancos de subida del reloj.
2. **Sumador de 4 bits con propagación de acarreo (`sumador4b`):** Sintetizado utilizando la librería estándar `NUMERIC_STD`, este componente se encarga de calcular matemáticamente el "próximo estado" sumando "0001" al estado actual.
3. **Multiplexor de Control (`mux_control`):** Opera como la lógica de control. Evalúa las señales de entrada (reset, load, enable) y selecciona qué valor debe ingresar al registro: un cero (para reset), el valor de la entrada externa `D` (para carga), el valor del sumador (para contar) o el valor actual (para mantener la cuenta).

A continuación, se presenta el código VHDL que implementa el *Top-Level* unificando los tres componentes sintetizados en un solo circuito secuencial sincrónico:

```vhdl
-- 1. SUMADOR
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD.all;

entity sumador4b is
    port (a, b : in STD_LOGIC_VECTOR (3 downto 0); y : out STD_LOGIC_VECTOR (3 downto 0));
end sumador4b;

architecture estructural of sumador4b is
begin
    y <= std_logic_vector(unsigned(a) + unsigned(b));
end estructural;
```

Figura 8. Código correspondiente al sumador de datos

```vhdl
-- 2. REGISTRO
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity registro4b is
    port (clk : in STD_LOGIC; d : in STD_LOGIC_VECTOR (3 downto 0); q : out STD_LOGIC_VECTOR (3 downto 0));
end registro4b;

architecture estructural of registro4b is
begin
    process (clk)
    begin
        if clk'event and clk = '1' then
            q <= d;
        end if;
    end process;
end estructural;
```

Figura 8.1. Código correspondiente al registro de datos

```vhdl
-- 3. MULTIPLEXOR
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux_control is
    port (reset, load, ending : in STD_LOGIC; d_in, sum_out, q_actual : in STD_LOGIC_VECTOR (3 downto 0); mux_out : out STD_LOGIC_VECTOR (3 downto 0));
end mux_control;

architecture estructural of mux_control is
begin
    mux_out <= "0000" when reset = '1' else
               d_in   when load = '1' else
               "0001" when (ending = '1' and q_actual = "1111") else
               sum_out when ending = '1' else
               q_actual;
end estructural;
```

Figura 8.2. Código correspondiente al multiplexor

```vhdl
-- 4. TOP-LEVEL
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity contador_top is
    port(clk, reset, load, ending : in STD_LOGIC; D : in STD_LOGIC_VECTOR(3 downto 0); Q : out STD_LOGIC_VECTOR(3 downto 0));
end contador_top;

architecture estructural of contador_top is
    component registro4b
        port (clk : in STD_LOGIC; d : in STD_LOGIC_VECTOR(3 downto 0); q : out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    component sumador4b
        port (a, b : in STD_LOGIC_VECTOR(3 downto 0); y : out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    component mux_control
        port (reset, load, ending : in STD_LOGIC; d_in, sum_out, q_actual : in STD_LOGIC_VECTOR(3 downto 0); mux_out : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal sum_out_sig, mux_out_sig, Q_internal : STD_LOGIC_VECTOR(3 downto 0);
begin
    U1_SUMADOR: sumador4b port map (a => Q_internal, b => "0001", y => sum_out_sig);
    U2_MUX: mux_control port map (reset => reset, load => load, ending => ending, d_in => D, sum_out => sum_out_sig, q_actual => Q_internal, mux_out => mux_out_sig);
    U3_REGISTRO: registro4b port map (clk => clk, d => mux_out_sig, q => Q_internal);

    Q <= Q_internal;
end estructural;
```

Figura 8.3. Código correspondiente al top level

4.1 Simulación y Diagrama de Tiempos

Para verificar el correcto funcionamiento del diseño sincrónico, se diseñó un banco de pruebas (contador_tb.vhd) que estimula las entradas de control y visualiza la respuesta del circuito mediante la herramienta GTKWave.

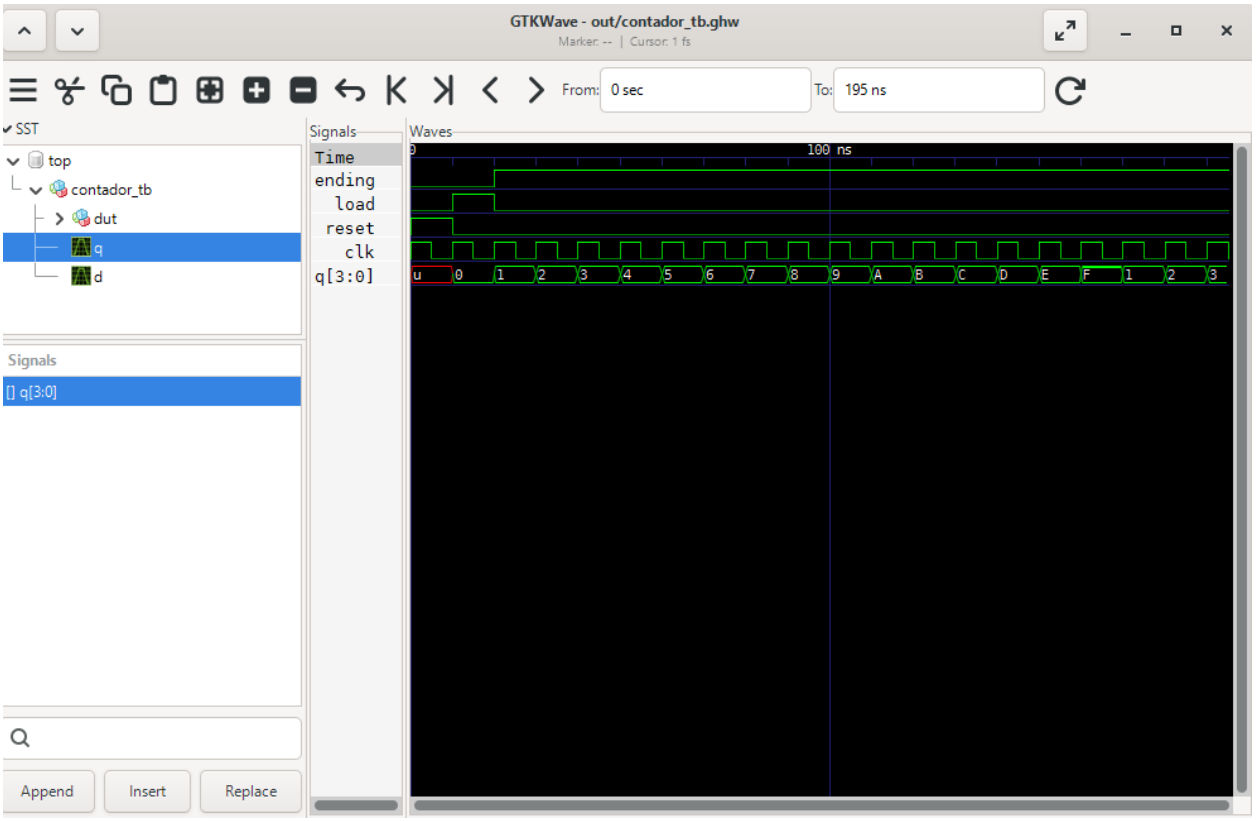


Figura 9. Diagrama de tiempos del contador binario de 4 bits (datapath) mostrando las operaciones sincrónicas de reset, carga paralela y habilitación de conteo repetitivo.

Como se observa en el diagrama de tiempos (Figura 8), el circuito cumple exitosamente con todos los requisitos. Al activarse la señal sincrónica de reset, la salida Q se inicializa. Cuando load se activa, la salida adopta el valor de D ("1") en el siguiente flanco de reloj. Finalmente, al activar ending (enable), el circuito cuenta progresivamente hasta 15 y retorna al valor inicial, demostrando el comportamiento adecuado del sumador y la lógica combinacional del multiplexor.

5 CONCLUSIONES

A lo largo del presente informe, se ha evidenciado de forma práctica cómo implementar sistemas secuenciales, particularmente contadores, empleando dos metodologías de diseño distintas: las Máquinas de Estado Finito (FSM) clásicas y la técnica de Datapath.

El diseño tradicional mediante FSM, basado en tablas de transición y mapas de Karnaugh, presenta la gran **ventaja** de generar circuitos altamente optimizados. Al encontrar las ecuaciones mínimas, se reduce al máximo el número de compuertas lógicas requeridas, lo que se traduce en un hardware más pequeño y económico. Sin embargo, su principal **desventaja** es la escalabilidad. A medida que aumenta el número de estados (por ejemplo, en un contador de más de 4 bits), el tamaño de las tablas de verdad crece exponencialmente, volviendo el cálculo algebraico manual y la construcción del diagrama de estados una tarea compleja y propensa a errores. Este método es más apropiado para circuitos de control simples y secuencias de pocos estados.

Por otro lado, la técnica de *Datapath* soluciona el problema de la escalabilidad al dividir el diseño en una ruta de datos y una lógica de control independiente. Su mayor **ventaja** es la modularidad y facilidad de diseño: permite reutilizar componentes MSI estándar (como sumadores o registros genéricos) en lugar de deducir ecuaciones bit a bit. En nuestro ensayo con el contador de 4 bits, bastó con conectar un sumador "+1" y un multiplexor para definir la secuencia y las condiciones de carga. Su **desventaja** radica en que los componentes modulares a menudo incluyen hardware "sobrante" que no siempre se utiliza al 100%, lo que puede resultar en un circuito final más grande o con mayor consumo energético que uno simplificado por Karnaugh. La técnica de Datapath es la más apropiada para el procesamiento de datos, para la

implementación de algoritmos aritméticos y para sistemas con una alta cantidad de registros, ya que prioriza la estructura conceptual clara sobre la minimización absoluta de compuertas.

6 REFERENCIAS

1. Harris, S. L., & Harris, D. (2022). *Digital Design and Computer Architecture: RISC-V Edition.* Morgan Kaufmann.
2. Wakerly, J. F. (2018). *Digital Design: Principles and Practices* (5th ed.). Pearson.
3. Floyd, T. L. (2015). *Digital Fundamentals* (11th ed.). Pearson.